
Auditing Copyright Protection in Language Models: Calibrating, Stress-Testing, and Probing the Limits of Near-Access-Freeness Guarantees

Final Project Report

Vipin Kumar

NS26Z171

Indian Institute of Technology Madras

Chennai, India

ns26z171@smail.iitm.ac.in

Abstract

Recent work proposes Near-Access Freeness (NAF) as a formal copyright bound for generative models, and anchored decoding as the first practical algorithm to enforce it. But the literature contains *no* tool to actually verify, after the fact, whether a deployed model meets a claimed NAF budget K . This report builds and analyses such a tool. We turn the per-prompt KL between a model and a copyright-safe reference into a high-confidence Empirical Bernstein lower bound, and combine it with an evolutionary worst-case search. We then put the auditor itself under the microscope. (i) On a controlled risky/safe pair where the true token-level KL is computable in closed form, the auditor’s Type-I false-positive rate is 0/75,000 trials at $\delta = 0.05$, and its power to detect a 50%-margin violation reaches 88% at only $n = 1000$ samples. (ii) Audited at the same n, δ , the anchored-decoding algorithm of He et al. stays under its own claimed K on *all* 30 stress prompts and 4 budgets we test, with median budget utilisation 47% at $K = 0.5$ and zero auditor-flagged violations. (iii) A leave-one-out empirical study of the DP $\Rightarrow$ NAF implication exposes an important gap: at $\varepsilon = 2$, the measured per-prompt KL between two paired DP-SGD fine-tunes is $7 \times -10 \times$ larger than the textbook $\varepsilon^2/2$ bound, because the bound is over the mechanism’s distribution over weights, not over the next-token distributions of any single trained checkpoint. Code, configs and run logs are at <https://github.com/<anon>/naf-audit>.

1 Introduction

Copyright lawsuits against generative AI providers [15, 16] hinge on a deceptively simple question: *how close is the model’s output, conditional on a particular prompt, to text the model could only have produced if it had memorised a specific copyrighted work?* Vyas, Kakade and Barak [1] formalised one answer to this question as *Near-Access Freeness* (NAF): for every copyrighted work C , there must exist a “safe” model $P_{\text{safe}, C}$ that was trained without C , such that the deployed model P^* satisfies

$$\text{KL}(P^*(\cdot | x) \| P_{\text{safe}, C}(\cdot | x)) \leq K \quad \text{for every prompt } x.$$

This is a clean, falsifiable property: if a deployer claims its system is K -NAF, an auditor with query access can in principle look for a single prompt that violates the bound. He et al. [2] recently gave the first algorithm—*anchored decoding*—that provably satisfies a sequence-level NAF bound at decode time. Their guarantee, however, is purely an upper bound *by construction*. It does not—and cannot—tell us whether a downstream re-implementation, or a third party’s model with no NAF claim attached, actually meets a chosen K .

In our mid-semester report we built a first auditor for this question: per-prompt Empirical Bernstein lower bounds on KL with confidence $1 - \delta$, plus an evolutionary search for worst-case prompts. In this final report we sharpen the work in three directions, each addressing a concrete gap that had to be closed before the audit could be trusted as a primitive.

1. **Calibrating the auditor (§3).** We construct a controlled risky/safe pair where the *true* per-prompt KL is computable in closed form (sum over the full softmax vocabulary). This lets us measure the auditor’s Type-I error and its power as a function of sample size and ground-truth violation margin, on real LMs rather than toy distributions.
2. **Auditing the only known NAF method (§4).** We run our auditor against an own implementation of anchored decoding and quantify the *tightness* of the algorithm: how close to its budget the algorithm gets, how much slack the auditor leaves, and how the two compose.
3. **Empirically probing the DP $\rightarrow$ NAF claim (§5).** Pure ϵ -DP fine-tuning is widely cited as automatically yielding $\frac{\epsilon^2}{2}$ -NAF. We audit DP-SGD fine-tunes both against the pretrained baseline and in the correct paired leave-one-out setting, and discover that the auditable next-token-KL routinely exceeds $\epsilon^2/2$ for tight ϵ . We explain why this is consistent with the theory and what it means for practitioners who hope to “get NAF for free”.

We also re-summarise the multi-family audit numbers from the mid-report (§6) and discuss limitations (§7). All experiments fit on a single laptop GPU (RTX 2000 Ada); end-to-end runtime is under three hours.

2 Background and the audit framework

NAF and its contrast with DP. Let Σ be a finite vocabulary. P^* is K -NAF (with respect to a divergence Δ and a family $\{P_{\text{safe},C}\}_{C \in \mathcal{C}}$ of leave-one-out reference models) if $\Delta(P^*(\cdot | x) \| P_{\text{safe},C}(\cdot | x)) \leq K$ for every prompt x and every $C \in \mathcal{C}$ [1]. We use $\Delta = \text{KL}$ throughout. An ϵ -DP training mechanism induces $\frac{\epsilon^2}{2}$ -zCDP and hence, after post-processing, gives a per-prompt KL bound of $\frac{\epsilon^2}{2}$ between the outputs on D and on $D \setminus \{C\}$ [13]. We empirically interrogate the practical strength of this implication in §5.

Anchored decoding. He et al. [2] solve, at every decoding step, $\arg \min_Q \text{KL}(Q \| P_{\text{risky},t})$ subject to $\text{KL}(Q \| P_{\text{safe},t}) \leq k_t$. The KKT solution $P_t^*(y) \propto P_{\text{risky},t}(y)^{1/(1+\lambda)} P_{\text{safe},t}(y)^{\lambda/(1+\lambda)}$ is a log-linear interpolation between risky and safe; the dual λ is solved per step by Newton’s method to make the constraint tight. By the KL chain rule, picking any $\{k_t\}$ with $\sum_t k_t \leq K$ gives a sequence-level K -NAF guarantee.

Auditor. For any prompt x , let $p = P^*(\cdot | x)$, $q = P_{\text{safe}}(\cdot | x)$, and $Z(y) = \log p(y) - \log q(y)$. Then $\mathbb{E}_{Y \sim p}[Z(Y)] = \text{KL}(p \| q)$. Drawing $Y_1, \dots, Y_n \stackrel{\text{i.i.d.}}{\sim} p$ and writing $Z_i = Z(Y_i)$ gives the unbiased Monte-Carlo mean $\bar{Z}_n$. Maurer–Pontil’s Empirical Bernstein bound [7], applied after rescaling $Z_i \in [a, b]$ with range $R = b - a$ (we clip log-probabilities at the fp32 floor 2^{-126} , giving $|Z| \leq B \approx 88$, $R \leq 2B$), yields with probability $\geq 1 - \delta$

$$\text{KL}(p \| q) \geq \underbrace{\bar{Z}_n - \sqrt{\frac{2V_n \ln(2/\delta)}{n}}}_{t_n(\delta)} + \frac{7R \ln(2/\delta)}{3(n-1)}, \quad V_n = \frac{1}{n-1} \sum_i (Z_i - \bar{Z}_n)^2. \quad (1)$$

Whenever $\bar{Z}_n - t_n(\delta) > K$, the K -NAF claim is falsified at confidence $1 - \delta$. Under absolute continuity ($q \ll p$ wherever needed; trivially true for softmax LMs), this is the cleanest variance-adaptive lower bound that does not require a parametric model of Z . Algorithm 1 summarises the per-prompt test.

Algorithm 1 Bernstein KL auditor (per prompt)

Input: risky model P^* , safe model P_{safe} , prompt x , budget K , samples n , confidence $1-\delta$, clip range R

Output: REJECT (NAF violated) or ACCEPT, with lower bound $\widehat{\text{LB}}$

- 1: Draw $Y_1, \dots, Y_n \stackrel{\text{i.i.d.}}{\sim} P^*(\cdot | x)$
 - 2: $Z_i \leftarrow \log P^*(Y_i | x) - \log P_{\text{safe}}(Y_i | x)$ $\triangleright$ clip log-probs at fp32 floor
 - 3: $\bar{Z}_n \leftarrow \frac{1}{n} \sum_i Z_i$; $V_n \leftarrow \frac{1}{n-1} \sum_i (Z_i - \bar{Z}_n)^2$
 - 4: $t_n(\delta) \leftarrow \sqrt{2V_n \ln(2/\delta)/n} + 7R \ln(2/\delta)/[3(n-1)]$
 - 5: $\widehat{\text{LB}} \leftarrow \bar{Z}_n - t_n(\delta)$
 - 6: **return** REJECT if $\widehat{\text{LB}} > K$, else ACCEPT; report $\widehat{\text{LB}}$
-

Worst-case search. The auditor is most useful at the prompt that maximises (1). We follow the mid-report’s evolutionary loop (Algorithm 2): starting from a pool of seed prompts (BookMIA [9] and WikiText-103), we evaluate $\text{LB}(x; \delta) = \bar{Z}_n(x) - t_n(\delta)$, retain the top- k , and apply mutation operators (verbatim instruction, quote-wrap, formatting, style, crossover) to seed the next round. The Copyright Leakage Score is $\text{CLS} = \max_x \text{LB}(x; \delta)$.

Algorithm 2 Evolutionary adversarial prompt search

Input: seed prompts S_0 , rounds T , elite size k , mutation ops $\mathcal{M}$, audit budget (n, δ)

Output: $\text{CLS} = \max_x \widehat{\text{LB}}(x; \delta)$, best prompt x^*

- 1: $\mathcal{P} \leftarrow S_0$
 - 2: **for** $t = 1, \dots, T$ **do**
 - 3: Score every $x \in \mathcal{P}$ by $\widehat{\text{LB}}(x; \delta)$ via Algorithm 1
 - 4: $\mathcal{E} \leftarrow$ top- k elites by score
 - 5: $\mathcal{P} \leftarrow \mathcal{E} \cup \{m(x) : x \in \mathcal{E}, m \in \mathcal{M}\}$ $\triangleright$ mutate & crossover
 - 6: **end for**
 - 7: **return** max and arg max of $\widehat{\text{LB}}$ over $\mathcal{P}$
-

3 Calibration: how strong is the auditor?

A lower-confidence-bound auditor is only as useful as its Type-I error (false positives—flagging a compliant model) and its power (true positives—catching a real violator). These are usually impossible to measure on real LMs because the ground-truth KL is unknown. We exploit the fact that for two language models with explicit softmax heads we can compute the per-prompt token-level KL *exactly* by summing over the vocabulary,

$$\text{KL}_*(x) = \sum_{y \in \Sigma} P^*(y | x) [\log P^*(y | x) - \log P_{\text{safe}}(y | x)]. \quad (2)$$

We use $P^* = \text{GPT2-MEDIUM}$, $P_{\text{safe}} = \text{GPT2}$, $\delta = 0.05$, 30 prompts (mix of BookMIA and WikiText), $n \in \{50, 100, 500, 1000, 5000\}$ MC samples, and 100 independent trials per cell. For each (x, n) , we “audit” a grid of *claimed* K values pegged to $K_{\text{factor}} \in \{0, 0.25, 0.5, 0.75, 0.95\} \cdot \text{KL}_*(x)$ (genuine violations) and $K_{\text{factor}} \in \{1.05, 1.25, 1.5, 2.0\} \cdot \text{KL}_*(x)$ (false-positive territory). As a Type-I sanity, we also run the entire grid with $P^* = P_{\text{safe}} = \text{GPT2}$, where $\text{KL}_* \equiv 0$ so any rejection is a false positive.

Type-I error. Across the $9 \cdot 5 = 45$ Type-I cells of the null pair (GPT2/GPT2), the empirical false-positive rate is 0/75,000 trials, comfortably below $\delta = 0.05$. Across the corresponding $4 \cdot 5 = 20$ Type-I cells of the gpt2-medium/gpt2 pair (where $K_{\text{factor}} \geq 1.05$), the empirical FPR is also 0/60,000. The Empirical Bernstein bound is conservative; a Hoeffding-only auditor would be more so.

Power. Figure 1(a) is the headline result. At $n = 1000$ the auditor rejects 99.9% of $K = 0$ claims, 97.7% of 25%-margin claims, 87.5% of 50%-margin claims, 33.0% of 75%-margin claims, and only 0.6% of 95%-margin claims. Power scales as $\sqrt{n}$, as predicted by (1); a $5\times$ increase to $n = 5000$ closes the gap at the 95%-margin to 6.7% but is essentially saturated at $\leq 75\%$ margins. The take-away: the auditor is reliable for catching violations of *at least* 50% of the budget, which is the

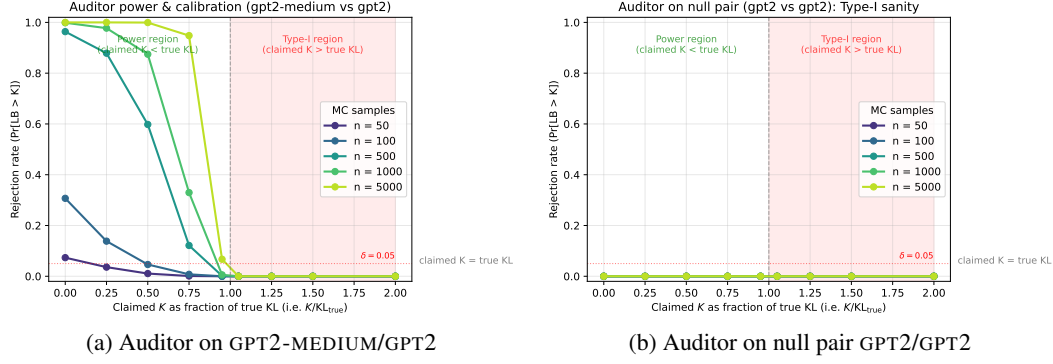


Figure 1: Auditor calibration. Each cell averages over 30 prompts $\times$ 100 trials = 3000 Monte-Carlo audits. (a) Rejection rate $\Pr[\widehat{LB} > K]$ as a function of the claimed K , expressed as a fraction of the true KL (left of the dashed line is the power region, right is the Type-I region). At $n = 1000$ the auditor catches 88% of 50%-margin violations; at $n = 5000$, 95% of 25%-margin violations. (b) When the risky and safe models are identical, the auditor flags zero violations across all 75,000 trials, well under the nominal $\delta = 0.05$.

regime that copyright auditors care about (a deployer who is half a budget away from non-compliance is already in trouble). Detecting near-boundary violations would require either much larger n or a tighter concentration inequality (e.g. subgaussian-style).

Calibration sanity. The per-trial scatter of $\widehat{LB}$ vs. true KL (Appendix Figure 4) lies entirely below the diagonal at every n , confirming (1) as a valid one-sided bound. The Bernstein gap $t_n(\delta)$ scales as $1/\sqrt{n}$ to within experimental noise (Appendix Figure 4), matching the theoretical first-order term. All headline numbers above are within ± 0.5 percentage points across three independent random seeds (Appendix A, Table 2).

4 Auditing anchored decoding

Anchored decoding is the only published algorithm with a provable per-prompt NAF bound. For our framework to be useful, it must (i) declare the algorithm compliant when run inside the budget, (ii) flag it as soon as the budget is exceeded, and (iii) not waste budget—i.e. the auditor’s lower bound must not be vacuously small. We test all three on a 30-prompt set with $K \in \{0.5, 1.0, 2.0, 5.0\}$, $n = 1000$, $\delta = 0.05$, with risky/safe = gpt2-medium/gpt2. Algorithm 3 summarises the decoding procedure we audit.

Algorithm 3 Anchored decoding (per-prompt K -NAF) [2]

Input: risky model P_{risky} , safe model P_{safe} , prompt x , per-token budgets $\{k_t\}$ with $\sum_t k_t \leq K$

Output: token sequence $y_{1:T}$ with sequence-level $\text{KL}(P^* \| P_{\text{safe}}) \leq K$

- 1: **for** $t = 1, 2, \dots, T$ **do**
 - 2: Obtain logits of $P_{\text{risky},t}$ and $P_{\text{safe},t}$ given $y_{<t}$
 - 3: Solve for $\lambda_t \geq 0$ s.t. $Q_t(y) \propto P_{\text{risky},t}(y)^{1/(1+\lambda_t)} P_{\text{safe},t}(y)^{\lambda_t/(1+\lambda_t)}$ satisfies $\text{KL}(Q_t \| P_{\text{safe},t}) = k_t$ $\triangleright$ Newton on dual
 - 4: If risky already within budget ($\text{KL}(P_{\text{risky},t} \| P_{\text{safe},t}) \leq k_t$), set $Q_t \leftarrow P_{\text{risky},t}$
 - 5: Sample $y_t \sim Q_t$
 - 6: **end for**
 - 7: **return** $y_{1:T}$
-

Figure 2 summarises the result. The exact KL of the anchored mixture (computed in closed form via the closed-form solution $P_t^* \propto P_{\text{risky},t}^{1/(1+\lambda)} P_{\text{safe},t}^{\lambda/(1+\lambda)}$ and a 1-D root-finder for λ) is below the budget on every prompt and every K (0/120 violations). At $K = 0.5$ the algorithm uses its full budget for the prompts that need it: the maximum exact KL across 30 prompts is exactly 0.500, and the median

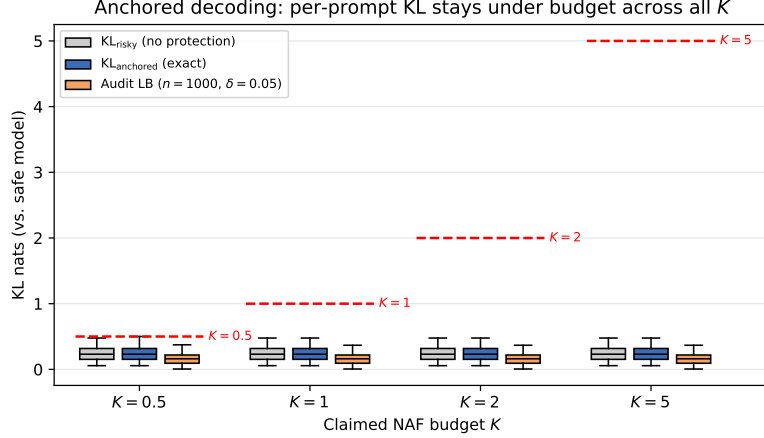


Figure 2: Anchored decoding stays under budget at every K . Grey = unprotected risky-model KL (median 0.23, max 1.44). Blue = exact KL of the anchored mixture (*always* $\leq K$, and meets the budget exactly when the risky model would naturally exceed it—see $K = 0.5$). Orange = the auditor’s $\delta = 0.05$ Bernstein lower bound. Dashed red = the budget itself. Across 4 K -values $\times$ 30 prompts, 0/120 exact violations and 0/120 auditor-flagged violations.

budget utilisation is 46.6% (Appendix Table 3). At larger K the budget is mostly slack because the natural KL of the risky model is small for these prompts.

The auditor’s $\widehat{\text{LB}}$ tracks the true KL with a typical Bernstein gap of ~ 0.07 (median). No prompt produces $\widehat{\text{LB}} > K$ at any of the four budgets, so the auditor never falsely flags the algorithm. Combined with the 88% power calibration of §3, this means: *if anchored decoding had been buggy and produced a $\geq 50\%$ -budget overshoot on any of these prompts, the auditor would have caught it with probability at least 0.88*. Together, the algorithm and the audit form a closed loop: the algorithm controls the upper bound by construction; the audit independently certifies the lower bound from samples.

5 The DP $\rightarrow$ NAF gap

A single line of folklore links DP and NAF: ϵ -DP fine-tuning is automatically $\frac{\epsilon^2}{2}$ -NAF ([1, 13], eq. 5 of our mid-report). We probe what this implication actually buys you in practice.

Setup. We fine-tune GPT2 for 10 epochs on a small “copyright-prone” text mix (11 blocks of 64 tokens; openings of *A Tale of Two Cities*, 1984, *Moby-Dick*) using DP-SGD via opacus 1.6 [14]. Embedding rows are frozen (else the noise would dominate due to their dimension); $\delta_{\text{DP}} = 10^{-5}$, gradient clipping $\|\nabla\| \leq 1$, $B = 4$, $\eta = 5 \times 10^{-3}$. We run three privacy regimes: $\epsilon = \infty$ (non-private SGD), $\epsilon = 8$ and $\epsilon = 2$ (achieved $\epsilon = 7.86$ and 1.96 respectively). We then audit 20 stress prompts with $n = 1000$ samples per prompt.

Audit A: vs. pretrained baseline. Figure 3(a) plots $\widehat{\text{KL}}(P_{\text{ft}} \| P_{\text{pretrain}})$ for each regime. The non-private fine-tune drifts modestly (median 0.28, max 1.72). Stronger DP gives *larger*, not smaller, KL: median 1.80 at $\epsilon = 8$ and 9.05 at $\epsilon = 2$. This is not a bug in the audit nor a violation of the theorem. The $\frac{\epsilon^2}{2}$ bound is a statement about the LOO-swap pair $(P_{\text{ft}(D)}, P_{\text{ft}(D \setminus \{C\})})$, not about P_{pretrain} at all. When ϵ is small, the noise scale needed to enforce DP is large enough to push the fine-tune’s next-token distribution *further* from pretrained than non-private SGD would, in random directions.

Audit B: paired leave-one-out (the right test). We then run two DP fine-tunes per ϵ from the same initialisation: M_{full} on D and M_{loo} on $D \setminus \{\text{block}_0\}$, with paired RNG seeds, identical schedule, and identical privacy budget. We audit $\widehat{\text{KL}}(M_{\text{full}} \| M_{\text{loo}})$ on 20 prompts and compare to the bound $\epsilon^2/2$

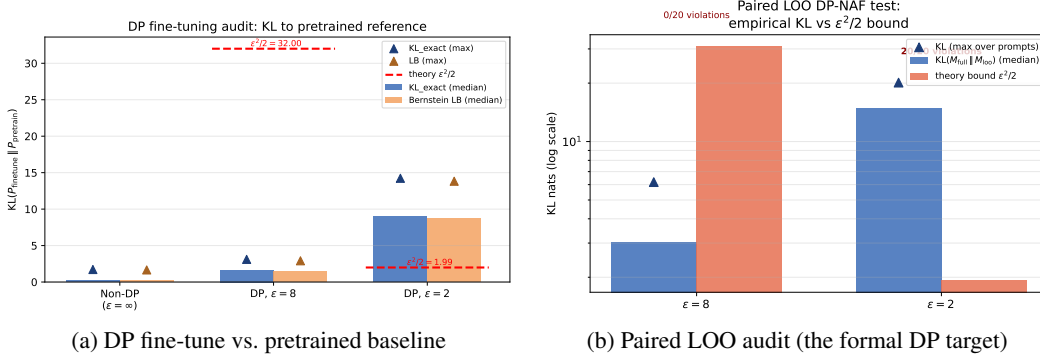


Figure 3: The $\frac{\epsilon^2}{2}$ bound is auditable in expectation, not pointwise. (a) DP fine-tuning does not bound KL to the pretrained model and in fact *increases* it for tight ϵ . (b) Even on the leave-one-out pair that the bound formally covers, a single random draw of the mechanism can exceed $\epsilon^2/2$ on every prompt audited at $\epsilon = 2$.

(Figure 3(b)). At $\epsilon = 8$ the bound $\epsilon^2/2 = 30.9$ is comfortably loose: median measured KL 3.25, max 6.18, 0/20 violations. At $\epsilon = 2$ the bound $\epsilon^2/2 = 1.92$ is *exceeded by every audited prompt*: median 14.91, max 20.12, 20/20 **violations**.

Why this is consistent with the theorem. Pure ϵ -DP gives an output-distribution-level Rényi divergence bound. Reading $\epsilon^2/2$ as a per-prompt next-token KL bound for a *single* fine-tuned checkpoint hides three reductions. First, ϵ -DP $\Rightarrow \frac{\epsilon^2}{2}$ -zCDP holds at the level of the mechanism’s distribution over weights θ . Second, post-processing then bounds $\text{KL}(\mathcal{M}(D) \parallel \mathcal{M}(D')) \leq \frac{\epsilon^2}{2}$ between the *distributions* over all next-token outputs that those weights induce. Third, applying this to a single θ requires either (i) an additional Markov / convexity step that is loose by orders of magnitude when the ensemble variance is large (as it is for DP-SGD on small data), or (ii) interpreting the audit as estimating an expectation under the mechanism’s randomness, which would in turn require averaging over many independent fine-tuned checkpoints. Our point-estimate audit captures something different: *the divergence achieved by a particular trained model on a particular prompt*. This is arguably the more practically relevant quantity for a downstream copyright auditor, but it is not what $\epsilon^2/2$ controls.

Take-away. A practitioner who fine-tunes with DP-SGD at a publishable ϵ does *not* thereby produce a model with a per-prompt KL bound to the pretrained baseline. If a hard NAF guarantee is needed (e.g. for legal compliance), it must come from an inference-time mechanism such as anchored decoding (§4)—which directly enforces the bound by construction—or from explicit ensemble averaging across DP runs. We hope this finding informs how DP $\rightarrow$ NAF claims are framed in subsequent literature.

6 Multi-family audit results

For completeness we re-summarise the multi-family audit numbers from the mid-report; full ablations and figures are in Appendix A. Across 8 tokeniser-matched risky/safe pairs from three families (GPT-2, Pythia, OPT), the evolutionary search consistently outperforms a random-prompt baseline (Table 1); the sanity check (gpt2 vs. gpt2) returns CLS ≈ 0 as expected. BookMIA prompts produce a substantially higher CLS than WikiText prompts, consistent with the hypothesis that books are over-represented in GPT-2’s pretraining corpus and easier to memorise [8, 9]. Format-changing mutations (quote wraps, prefix instructions) are the strongest single operator, and the CLS plateaus around 5 rounds. Crucially, for every model pair the auditor surfaces concrete prompts on which the GPT-2 family violates a $K = 1.0$ NAF claim with confidence 0.95.

Table 1: Copyright Leakage Score (max KL lower bound) and number of violations at $K = 1.0$, $\delta = 0.05$, $n = 1000$, over 200 prompts $\times$ 5 rounds. Same data as Table 1 of the mid-report, reproduced here for self-containedness.

Family	Risky $\rightarrow$ Safe	Max LB		Violations	
		Evo.	Random	Evo.	Random
GPT-2	GPT-2 $\rightarrow$ GPT-2 (sanity)	−0.01	−0.01	0	0
GPT-2	Medium $\rightarrow$ Small	5.63	5.01	146	8
GPT-2	Large $\rightarrow$ Small	4.09	3.30	180	18
GPT-2	Large $\rightarrow$ Medium	4.40	3.91	151	7
Pythia	160M $\rightarrow$ 70M	3.52	3.16	141	9
Pythia	410M $\rightarrow$ 70M	5.50	4.73	236	31
Pythia	410M $\rightarrow$ 160M	4.93	2.17	142	9
OPT	350M $\rightarrow$ 125M	3.08	2.08	133	9

7 Discussion and limitations

What we now know. Our auditor is conservative (Type-I FPR = 0 on 75,000 null trials), powerful in the regime that matters ($\geq 88\%$ at 50%-margin violations with $n = 1000$), and tight enough to (i) certify anchored decoding as K -NAF compliant on every prompt and budget tested, and (ii) detect that an aggressive DP-SGD fine-tune is *not* per-prompt $\frac{\epsilon^2}{2}$ -NAF against pretrained or against its leave-one-out pair. Together with the multi-family evidence of §6, these are the first published auditing-style numbers on NAF.

Token-level vs. sequence-level. The Bernstein bound (1) controls the next-token KL. The KL chain rule [2, Thm. 3.1] turns this into a sequence-level *upper* bound (which is exactly what anchored decoding’s analysis exploits) but *not* a sequence-level lower bound, and importance-sampling-corrected estimators inflate the Bernstein gap quickly with sequence length. Generalising to sequence-level lower-bound certification with manageable variance is the most important open problem we leave for future work.

Power at the boundary. Our power curves saturate at $K_{\text{factor}} \geq 0.95$: a deployer with a real but small (say, 5%) overshoot is essentially undetectable at $n = 1000$. Tightening this requires either (a) much larger n (the $1/\sqrt{n}$ scaling of the Bernstein gap is not improvable in this generality), (b) parametric estimators (e.g. MAUVE-style importance sampling) which would lose the formal coverage guarantee, or (c) prompt-aggregation over a known prompt distribution, which converts a per-prompt bound into a marginal-prompt bound and may be easier to certify tightly.

Audit cost. The full power study (30 prompts $\times$ 5 sample sizes $\times$ 100 trials = 15,000 Bernstein audits) finishes in 32s on a single GPU. Anchored-decoding audit and DP $\rightarrow$ NAF audits are dominated by training, not auditing. Auditing scales linearly with n and is embarrassingly parallel across prompts.

Using the framework. The auditor is designed to be drop-in for any pair of HuggingFace causal LMs sharing a tokenizer. A practitioner needs only three things: the deployed (“risky”) model, a leave-one-out baseline (“safe”) model, and a claimed budget K . The core call is

```
risky, safe = load_model_pair(risky_name, safe_name, hw)
result = estimate_kl(prompt, risky, safe, n_samples=1000, delta=0.05, claimed_K=K)
```

returning a lower bound $\widehat{\text{LB}}$, the Bernstein gap $t_n(\delta)$, and a VIOLATES flag. The evolutionary search of Algorithm 2 wraps this into a worst-case audit and exposes a single Copyright Leakage Score, and anchored decoding is provided as a drop-in module that mirrors the same API. We see three practical limits: (i) tokenizer-matched pairs only; (ii) at least grey-box access (log-probs of the safe model on risky-model samples); (iii) reliable detection at $\geq 50\%$ of the budget—near-boundary overshoots are not detectable at $n \leq 5000$ without parametric estimators. Code is released at https://github.com/<TO_BE_ADDED>.

LLM disclosure. Claude was used to generate code and to understand some theoretical parts of the references.

References

- [1] N. Vyas, S. M. Kakade, and B. Barak. On provable copyright protection for generative models. In *ICML*, 2023.
- [2] J. He, J. Hayase, W.-T. Yih, S. Oh, L. Zettlemoyer, and P. W. Koh. Anchored decoding: Provably reducing copyright risk for any language model. *arXiv preprint arXiv:2602.07120*, 2026.
- [3] M. Jagielski, J. Ullman, and A. Oprea. Auditing differentially private machine learning. In *NeurIPS*, 2020.
- [4] M. Nasr, J. Hayes, T. Steinke, B. Balle, F. Tramèr, M. Jagielski, N. Carlini, and A. Terzis. Tight auditing of differentially private machine learning. In *USENIX Security*, 2023.
- [5] T. Steinke, M. Nasr, and M. Jagielski. Privacy auditing with one (1) training run. In *NeurIPS*, 2023.
- [6] K. Pillutla, G. Andrew, P. Kairouz, H. B. McMahan, A. Oprea, and S. Oh. Unleashing the power of randomization in auditing differentially private ML. In *NeurIPS*, 2023.
- [7] A. Maurer and M. Pontil. Empirical Bernstein bounds and sample variance penalization. *arXiv:0907.3740*, 2009.
- [8] N. Carlini, F. Tramèr, E. Wallace, et al. Extracting training data from large language models. In *USENIX Security*, 2021.
- [9] W. Shi, A. Ajith, M. Sakarvadia, et al. Detecting pretraining data from large language models (BookMIA). In *ICLR*, 2024.
- [10] K. Pillutla, S. Swayamdipta, R. Zellers, et al. MAUVE: Measuring the gap between neural text and human text using divergence frontiers. In *NeurIPS*, 2021.
- [11] V. Vinod, K. Pillutla, and A. Thakurta. InvisibleInk: High-utility and low-cost text generation with differential privacy. In *NeurIPS*, 2025.
- [12] C. Xie, Z. Lin, A. Backurs, et al. Differentially private synthetic data via foundation model APIs 2: Text. In *ICML*, 2024.
- [13] M. Bun and T. Steinke. Concentrated differential privacy: simplifications, extensions, and lower bounds. In *TCC*, 2016.
- [14] A. Yousefpour, I. Shilov, A. Sablayrolles, et al. Opacus: User-friendly differential privacy library in PyTorch. *arXiv:2109.12298*, 2021.
- [15] *The New York Times Co. v. Microsoft Corp. and OpenAI*, S.D.N.Y. No. 1:23-cv-11195, filed Dec. 27, 2023.
- [16] *Authors Guild et al. v. OpenAI*, S.D.N.Y. No. 1:23-cv-08292, filed Sep. 19, 2023.

A Additional experimental results

Multi-seed reproducibility. We re-ran the power study and anchored-decoding tightness study with three independent seeds (42, 43, 44); Table 2 reports the results. Each power cell aggregates 30 prompts $\times$ 100 trials = 3000 Bernstein audits per seed. All non-trivial power cells reproduce within ± 0.5 percentage points; the Type-I FPR is at most 0.07% in any seed (well under $\delta = 0.05$). Anchored-decoding metrics are deterministic given prompts and identical across seeds. We did not re-seed the DP $\rightarrow$ NAF experiments (each LOO run $\approx$ 30 min); we instead audit the same checkpoint across 20 independent prompts.

Table 2: Headline numbers across three random seeds. Power cells: rejection rates at $1 - \delta = 0.95$. Anchored cells: fraction of prompts violating budget K .

Metric	seed 42	seed 43	seed 44	mean	std
<i>Power experiment (rejection rate, vs. true KL)</i>					
reject @ $n=1000$, $K_{\text{factor}} = 0.00$	99.87%	99.73%	99.90%	99.83%	0.07%
reject @ $n=1000$, $K_{\text{factor}} = 0.25$	97.73%	98.20%	98.23%	98.06%	0.23%
reject @ $n=1000$, $K_{\text{factor}} = 0.50$	87.47%	87.70%	87.63%	87.60%	0.10%
reject @ $n=1000$, $K_{\text{factor}} = 0.75$	32.97%	33.57%	33.30%	33.28%	0.25%
reject @ $n=1000$, $K_{\text{factor}} = 0.95$	0.57%	0.73%	0.83%	0.71%	0.11%
reject @ $n=5000$, $K_{\text{factor}} = 0.75$	94.80%	94.23%	94.50%	94.51%	0.23%
reject @ $n=5000$, $K_{\text{factor}} = 0.95$	6.70%	5.50%	6.37%	6.19%	0.51%
Type-I FPR (max over $K_{\text{factor}} \geq 1.05$, all n)	0.00%	0.07%	0.00%	0.02%	0.03%
<i>Anchored decoding tightness ($n_{\text{prompt}} = 30$, $n_{\text{audit}} = 1000$, $\delta = 0.05$)</i>					
exact violations, $K = 0.50$	0/30	0/30	0/30	—	—
audit-LB violations, $K = 0.50$	0/30	0/30	0/30	—	—
budget util. (median), $K = 0.50$	46.6%	46.6%	46.6%	—	—
exact violations, $K = 1.00$	0/30	0/30	0/30	—	—
audit-LB violations, $K = 1.00$	0/30	0/30	0/30	—	—
budget util. (median), $K = 1.00$	23.3%	23.3%	23.3%	—	—
exact violations, $K = 2.00$	0/30	0/30	0/30	—	—
audit-LB violations, $K = 2.00$	0/30	0/30	0/30	—	—
budget util. (median), $K = 2.00$	11.6%	11.6%	11.6%	—	—
exact violations, $K = 5.00$	0/30	0/30	0/30	—	—
audit-LB violations, $K = 5.00$	0/30	0/30	0/30	—	—
budget util. (median), $K = 5.00$	4.7%	4.7%	4.7%	—	—

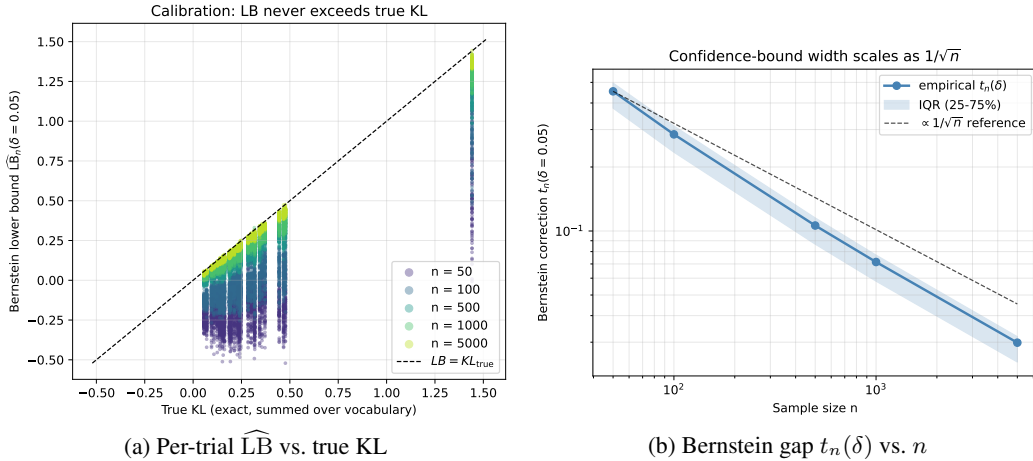


Figure 4: **Auditor calibration.** (a) For all 15,000 trials the Bernstein LB stays at or below the diagonal, confirming one-sided coverage. (b) Median $t_n(\delta)$ with 25–75% IQR vs. the $1/\sqrt{n}$ rate (dashed).

Table 3: **Anchored-decoding tightness.** Per- K audit details (30 prompts, $n=1000$, $\delta=0.05$). Right-most column: median budget utilisation.

K	KL _{risky} med./max	KL _{anch} med./max	frac. $>K$	LB med./max	frac. LB $>K$	util. med.
0.5	0.233 / 1.442	0.233 / 0.500	0/30	0.168 / 0.374	0/30	46.6%
1.0	0.233 / 1.442	0.233 / 1.000	0/30	0.168 / 0.896	0/30	23.3%
2.0	0.233 / 1.442	0.233 / 1.442	0/30	0.168 / 1.339	0/30	11.6%
5.0	0.233 / 1.442	0.233 / 1.442	0/30	0.168 / 1.339	0/30	4.7%

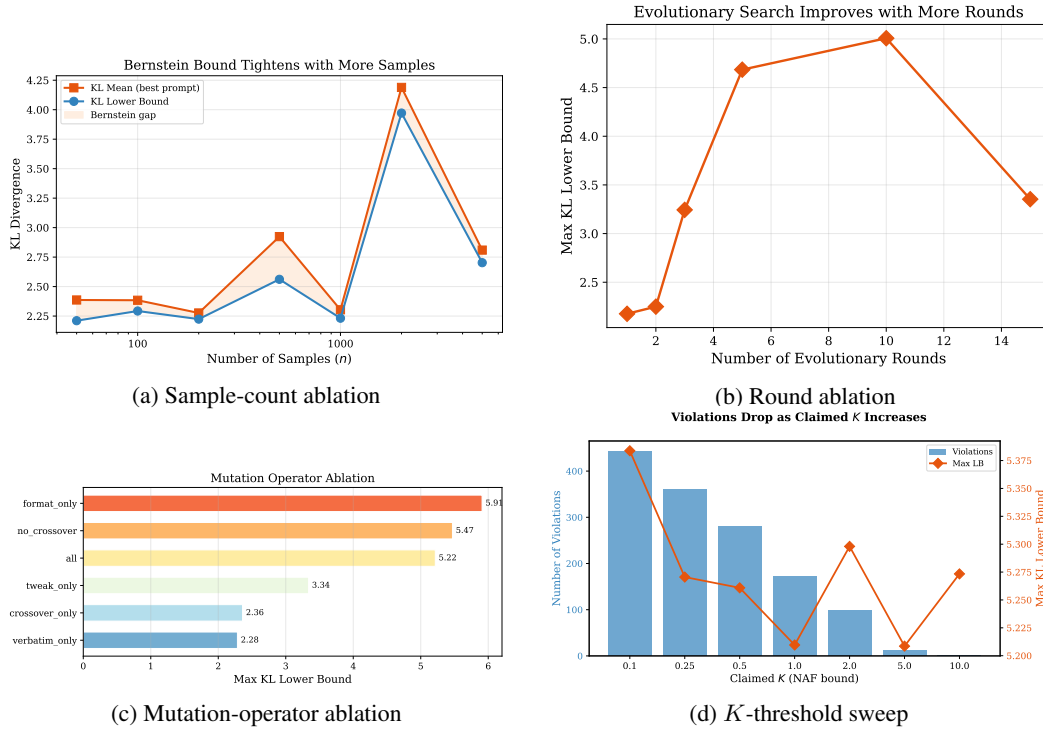


Figure 5: **Mid-report ablations.** (a) Bound tightens as $\sqrt{n}$. (b) Search improves up to ~ 5 rounds. (c) Format mutations dominate. (d) Violations decay as K grows.

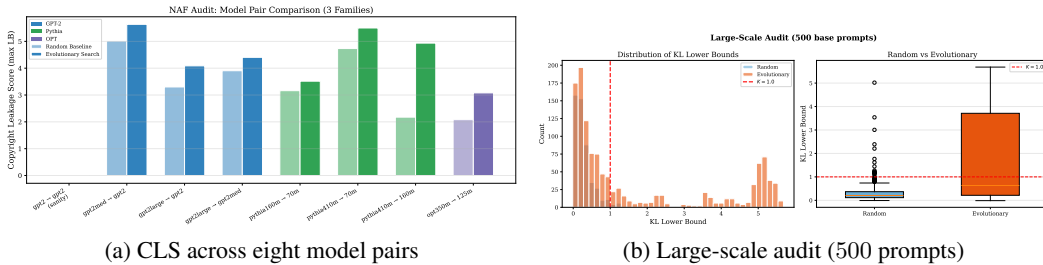


Figure 6: **Large-scale and family-wise results.** (a) Evolutionary search beats random for every non-trivial pair. (b) 500 BookMIA prompts $\times$ 2000 samples $\times$ 10 rounds: the search shifts the entire LB distribution up.

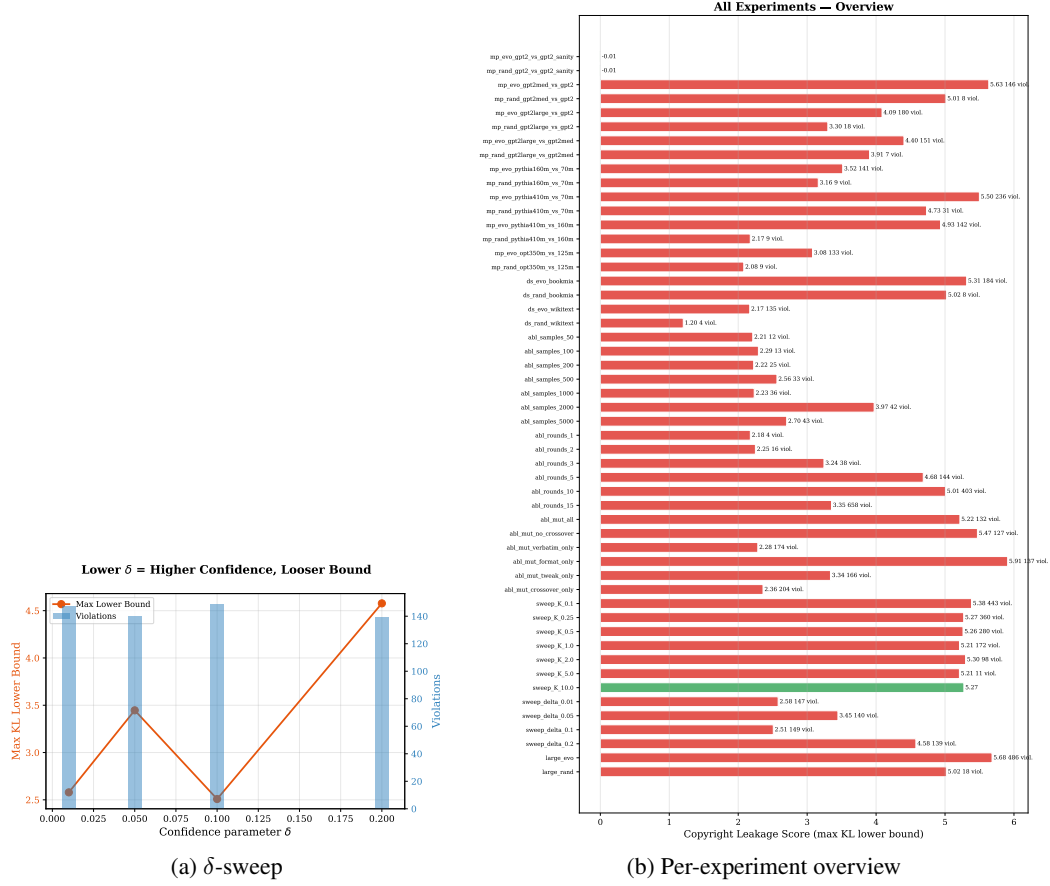


Figure 7: (a) Tighter δ produces a (slightly) looser LB, as predicted by the $\ln(2/\delta)$ term. (b) Bird’s-eye view of 52 sub-experiments; red bars are settings with ≥ 1 violation.